硬件与Agent · 增量学习 06/07

AI辅助硬件入门：把验收写进每一步

少写代码不等于少做核查；从合成备份练恢复纪律

待验证

原经验帖与其引用长教程已读；合成文件备份、损坏拒收和恢复真实完成。没有真实板子识别、固件备份或烧录。

先记住

这是原帖作者经验性表达，没有定义任务样本、耗时或成功率，本批不把它当测量。AI能读文档、生成命令并解释日志，但接线、型号核对、授权与物理安全仍需要真实证据。

原帖：https://x.com/i/status/2112021182568632361
访问：2026-11-11（Asia/Shanghai）
知识库main：a8674777e18302b8804b137212b1396cbfe5a747

本册10个递进问题、10幅内联SVG、4个用途、4步可复跑实验。数字分清：作者报告 / 本批真实执行 / 教学合成 / 尚待验证。

Q01 AI真的能完成80%的工作吗?

这是原帖作者经验性表达，没有定义任务样本、耗时或成功率，本批不把它当测量。AI能读文档、生成命令并解释日志，但接线、型号核对、授权与物理安全仍需要真实证据。

作者经验 → 拆分任务 → 实证验收

Q02 对AI说什么才不容易跑偏?

把一句需求写成目标、禁止事项、可观察输出与通过条件。例如先只读识别，不写Flash；回报芯片、容量、串口和证据。引用教程提供分步提示词，不能从能生成命令推导命令在你的板子上正确。

目标 → 边界 → 证据与验收

Q03 成品板就完全不用懂电路吗?

板载外设减少接线，但电源、电平、驱动和板型仍影响安全与可用性。原帖也提醒裸开发板要接线基础；本批不让没有基础的人接市电、改锂电池或把5V施加到不容忍的GPIO。

板载集成 → 仍有电气约束 → 先查原理图

Q04 为什么第一步只读识别?

同名芯片可能对应不同Flash容量和板载外设。先定位真实设备和兼容固件，再备份，最后才考虑写入。本批没有USB设备，因此只执行文件层实验，不虚构串口和芯片检测输出。

确认设备 → 确认兼容 → 再考虑写入

Q05 备份成功应看什么?

至少看读取是否无错、文件大小是否覆盖预期区域、哈希是否保存，以及恢复方法是否适用。合成4096字节镜像保存到独立backup.bin并逐字节相等；4096只是练习大小，不是ESP32容量。

完整读取 → 大小与哈希 → 恢复预案

Q06 SHA-256能证明固件安全吗?

不能。它证明字节是否与期望副本一致；如果镜像和期望哈希一起来自不可信来源，恶意内容同样可通过。来源真实性要靠可信发布、签名与版本核验，运行安全还需额外测试。

字节一致 → 可信来源 → 运行行为

Q07 损坏一个字节为什么要拒绝?

固件、资源和分区内容对字节敏感。练习把偏移100的一个字节翻转，哈希由c8f5…变成0f1b…，校验门输出REJECT。它演示完整性检测，不证明真实芯片会如何启动。



Q08 恢复时为什么不能覆盖备份?

备份是回到旧状态的最后证据；应写到候选或目标副本，再单独核对恢复后哈希。本练习复制backup到restored.bin，没有覆盖backup，也没有接触任何真实设备。



Q09 错误日志应该只留成功那次吗？

不应该。要保存首次失败、原因、修改点和复跑结果，避免后续把修复经验丢掉。本批第7项C主机测试首次缺少日志桩头文件，失败仍在日志中，补齐官方include后才通过。

首次失败 → 最小修复 → 复跑证据

Q10 怎么结束一轮而不是无限折腾？

预先定义有限验收，例如指定文件大小、哈希一致、反例拒收、恢复相等，逐项保存日志。本批不勾选用户Reminder，不创建日程；文件实验结束后只把真实硬件环节标待验证，不谎称板子已完成。

有限标准 → 落盘结果 → 边界清楚

可以用在哪里?

1. 给AI一条包含目标、禁止事项、输出证据和验收条件的请求。
2. 在真实设备前，用无害文件熟悉备份与校验工作流。
3. 发现一个字节改动也能被校验门拒绝，理解一致性边界。
4. 建立故障定位顺序，区分线材、端口、驱动、板型和程序。

与旧知识的差异

只补充现有规格书先行、护栏和证据优先概念的硬件学习实例；不把80%经验说法当量化结果，也不复制旧MCP基础概念。

不越过的证据边界

原经验帖与其引用长教程已读；合成文件备份、损坏拒收和恢复真实完成。没有真实板子识别、固件备份或烧录。

完整来源见 sources.md；真实命令与首次失败见 experiment-log.md。代码压缩包包含复跑入口、受控输入和必要来源说明。

主实验 · 前半程

1. 声明合成镜像

目的：输出实验范围和Python版本。

```
cd /workspace/reminder-batch-2026-11-11/item06/exercise
python3 lab.py 1
```

真实输出（关键字段）

{"python":"3.13.5","scope":"合成4096字节镜像；不是实际ESP32固件"}

验证：scope写明4096字节且非真实固件。

2. 创建并备份镜像

目的：用可重复字节序列生成文件、哈希与副本。

```
cd /workspace/reminder-batch-2026-11-11/item06/exercise
python3 lab.py 2
```

真实输出（关键字段）

{"bytes":4096,"sha256":"c8f5d0341d54d951a71b136e6e2afcb14d11ed8489a7ae126a8fee0df6ecf193","backup_equal":true}

验证：bytes=4096，backup_equal=true。

主实验 · 后半程

3. 注入单字节损坏

目的：只改隔离候选文件并比较哈希。

```
cd /workspace/reminder-batch-2026-11-11/item06/exercise
python3 lab.py 3
```

真实输出（关键字段）

{"corrupt_sha256":"0f1b80d16728f1e0d37733f6aa593bb9a75c4fc76d176e72d15e0829142adc6d","expected_sha256":"c8f5d0341d54d951a71b136e6e2afcb14d11ed8489a7ae126a8fee0df6ecf193","gate":"REJECT","changed_bytes":1}

验证：gate=REJECT，changed_bytes=1。

4. 从备份恢复候选

目的：写restored.bin并重新计算哈希。

```
cd /workspace/reminder-batch-2026-11-11/item06/exercise
python3 lab.py 4
```

真实输出（关键字段）

{"restored_sha256":"c8f5d0341d54d951a71b136e6e2afcb14d11ed8489a7ae126a8fee0df6ecf193","restored":true,"physical_flash":"未执行","note":"校验值证明一致性，不证明来源真实性/设备可启动"}

验证：restored=true，physical_flash=未执行。

来源与版本账本

原帖全文已读：https://x.com/i/status/2112021182568632361

引用：https://x.com/i/status/2110183197850116257

78/xiaozhi-esp32
SHA a73e9a7e266e3196d1368ba46c30bee0d81f0a64
https://github.com/78/xiaozhi-esp32/tree/a73e9a7e266e3196d1368ba46c30bee0d81f0a64

知识库查重 SHA

a8674777e18302b8804b137212b1396cbfe5a747

已读总览、接缝、级联语音、证据优先与Muse SDK笔记。样式沿用仓库青蓝卡片、大编号与橙色强调。

详细文件指纹、不可读URL及媒体未观看限制见 sources.md。作者性能不能写成本批实测；练习成功不能写成硬件已完成。

状态：待验证
原经验帖与其引用长教程已读；合成文件备份、损坏拒收和恢复真实完成。没有真实板子识别、固件备份或烧录。